

Project Setup & Deployment Guide

Project GitHub Repository

📌 GitHub Link:

https://github.com/yashprogrammer/ReAct_agent.git

Video 2 — Environment Setup

Installing UV (Universal Python Package Manager)

macOS

Method 1 — Using Homebrew (Recommended)

If Homebrew is already installed:

```
brew install uv
```

If Homebrew is **not installed**, install it first:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then install UV:

```
brew install uv
```

Method 2 — Using the Official Installer

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

Windows

Method 1 — Using PowerShell (Recommended)

Open **PowerShell** and run:

```
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

This is the **official installer from Astral**, the makers of **UV**, **Ruff**, and **Rye**.

Method 2 — Using pip (Fallback)

```
pip install uv
```

Works fine, but the official installer is recommended.

Linux (Ubuntu / Debian / Arch / Fedora)

Method 1 — Using Official Installer (Recommended)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

UV Project Setup

Open **Terminal / Command Prompt** inside the project directory and run:

```
uv init
```

Then install required dependencies:

```
uv add langchain-core langchain-openai python-dotenv langchain-google-genai
```

Video 4 — Building a Simple ReAct Agent

Install additional dependencies:

```
uv add langchain langchain-groq
```

 **Reference Documentation:**

<https://docs.langchain.com/oss/python/langchain/agents>

Video 5 — Adding First Tool (Dummy Tool)

 **LangChain Tools Documentation:**

<https://docs.langchain.com/oss/python/langchain/tools>

This video introduces how tools are defined and used by a ReAct agent.

Video 6 — Adding Weather Tool

Geo Coding API

<https://geocoding-api.open-meteo.com/v1/search?name={city}&count=1&language=en&format=json>

Weather API

https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}¤t=temperature_2m,weather_code

These APIs are used to fetch real-time location and weather data.

Video 7 — Notion Setup

 **Project GitHub Repository:**
https://github.com/yashprogrammer/ReAct_agent.git

 **Notion Database Setup Script Path:**

/scripts/setup_notion_databases.py

This script automatically creates required Notion databases.

Deployment Guide (AWS EC2 + CI/CD)

1. Prerequisite — GitHub Repository

Ensure the complete project code is pushed to a GitHub repository.

2. EC2 Setup

- **OS:** Ubuntu 22.04 LTS (Recommended)
 - **Instance Type:**
 - **t2.micro** (Free Tier)
 - Use **t3.small** if memory issues occur
-

Security Group Configuration

- ✓ **SSH (Port 22)** — Allow from **0.0.0.0/0**
 - ✓ **Custom TCP (Port 8000)** — Allow from **0.0.0.0/0** (**Critical**)
-

Install Docker on EC2

SSH into your EC2 instance and run:

```
sudo apt update
sudo apt install -y docker.io
sudo usermod -aG docker $USER
```

➡ **Log out and log back in** for group changes to take effect.

3. Network Configuration (CRITICAL STEP)

Your application runs on **port 8000**, which is blocked by default.

Steps to Open Port 8000:

1. Go to **EC2 Dashboard**
 2. Select your instance
 3. Open the **Security** tab
 4. Click the **Security Group**
 5. Click **Edit inbound rules**
 6. Add a rule:
 - Type: Custom TCP
 - Port: 8000
 - Source: Anywhere-IPv4 (0.0.0.0/0)
 7. Save rules
-

4. GitHub Secrets Configuration

Go to:

GitHub → **Repository** → **Settings** → **Secrets and variables** → **Actions**

Infrastructure Secrets

- **HOST** → Public IP of EC2 instance
 - **USERNAME** → Usually **ubuntu**
 - **SSH_KEY** → Private SSH key (.pem file content)
 - **CR_PAT** → GitHub Personal Access Token (scope: **read:packages**)
-

Application Secrets

- `GOOGLE_API_KEY`
- `GROQ_API_KEY`
- `NOTION_API_KEY`
- `NOTION_NOTES_DB_ID`
- `NOTION_CALENDAR_DB_ID`

5. Deploy

- Push code to the **main branch**
- Open **GitHub** → **Actions**
- Monitor CI/CD pipeline execution and deployment status

✅ Your ReAct Agent is now deployed and accessible via EC2.

6. Terminate EC2 instance from EC2 dashboard (IMPORTANT)

Do this after you have successfully deployed your project, if you don't want to **bear any cost and exhaust any free credits**

The screenshot displays the AWS Management Console's EC2 Instances page. On the left, a navigation sidebar lists various EC2-related services. The main content area shows a table of instances. One instance, named 'Test' with ID 'i-0e24e828132b066d5', is in a 'Running' state. An 'Actions' dropdown menu is open for this instance, highlighting the 'Stop instance' option. Below the table, the 'Details' tab for the selected instance is shown, providing information such as the Instance ID, Public IPv4 address (3.80.173.233), Private IPv4 addresses (172.31.2.211), and the current state (Running).